Exercício 9.1: ServiceAccount somente-leitura validada com can-i

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1, namespace `store-dev`.
- Sequência completa em `03-commands.sh`.

Manifestos

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: monitor-sa
  namespace: store-dev
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: leitura-store
  namespace: store-dev
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log", "services", "endpoints", "configmaps"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: leitura-store-monitor
  namespace: store-dev
subjects:
  - kind: ServiceAccount
    name: monitor-sa
    namespace: store-dev
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: leitura-store
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: monitor
  namespace: store-dev
spec:
  serviceAccountName: monitor-sa
  containers:
    - name: curl
      image: curlimages/curl:8.10.1
      command: ["sleep", "3600"]
```

(a) `can-i` com impersonation

```
$ AS=system:serviceaccount:store-dev:monitor-sa
$ kubectl auth can-i <verbo> <recurso> -n store-dev --as=$AS
get pods                    yes
list services               yes
watch endpoints             yes
get configmaps              yes
get secrets                 no
create pods                 no
delete pods                 no

$ kubectl auth can-i get pods --subresource=log -n store-dev --as=$AS
yes
$ kubectl auth can-i list pods -n default --as=$AS
no
```

São nove combinações: cinco `yes` dentro do que foi concedido e quatro `no`. Duas delas são as pedidas (`get secrets` e `create pods`). As outras duas são uma escrita adjacente (`delete pods`) e o mesmo verbo em outro namespace (`default`), que mostra que a Role vale só em `store-dev`. Para `pods/log` foi usado `--subresource=log`, porque `can-i get pods/log` pergunta sobre um Pod chamado `log` (ver o exercício 9.3).

```
$ kubectl auth can-i --list -n store-dev --as=$AS
Resources                                       Non-Resource URLs     Resource Names   Verbs
selfsubjectreviews.authentication.k8s.io        []                    []               [create]
selfsubjectaccessreviews.authorization.k8s.io   []                    []               [create]
selfsubjectrulesreviews.authorization.k8s.io    []                    []               [create]
configmaps                                      []                    []               [get list
watch]
endpoints                                       []                    []               [get list
watch]
pods/log                                        []                    []               [get list
watch]
pods                                            []                    []               [get list
watch]
services                                        []                    []               [get list
watch]
                                                [/api/*]              []               [get]
                                                [/healthz]            []               [get]
                                                ... (demais URLs não-recurso de discovery e saúde)
```

Os recursos com nome são exatamente os cinco da Role. As três primeiras linhas (`self*reviews`) e as URLs não-recurso não vêm da Role: vêm das ClusterRoles padrão

`system:basic-user`, `system:discovery` e `system:public-info-viewer`, ligadas a todo usuário autenticado. Sem elas, nem o próprio `can-i` funcionaria.

(b) Token e payload JWT

```
$ kubectl create token monitor-sa -n store-dev --duration=1h | cut -d. -f2 | tr '_-' '/+' | base64 -d
{
    "aud": ["https://kubernetes.default.svc.cluster.local"],
    "exp": 1792817325,
    "iat": 1792813725,
    "iss": "https://kubernetes.default.svc.cluster.local",
    "jti": "23854204-36be-40e7-bdbc-bd033bc1586d",
    "kubernetes.io": {
        "namespace": "store-dev",
        "serviceaccount": {"name": "monitor-sa", "uid": "d21fa65c-0473-4b77-9488-e550095f1873"}
    },
    "nbf": 1792813725,
    "sub": "system:serviceaccount:store-dev:monitor-sa"
}

$ date -d @1792813725   →  sáb. 24 out 2026 00:48:45 -03   (iat)
$ date -d @1792817325   →  sáb. 24 out 2026 01:48:45 -03   (exp)
```

- `sub` é exatamente `system:serviceaccount:store-dev:monitor-sa`, a mesma string usada no `--as`.
- `aud` é o próprio API server. Outro serviço que valide a audiência rejeita esse token.
- `exp - iat = 3600 s`, ou seja, o `--duration=1h`.

O `tr '_-' '/+'` converte base64url (o formato do JWT) para base64 comum antes do `base64 -d`.

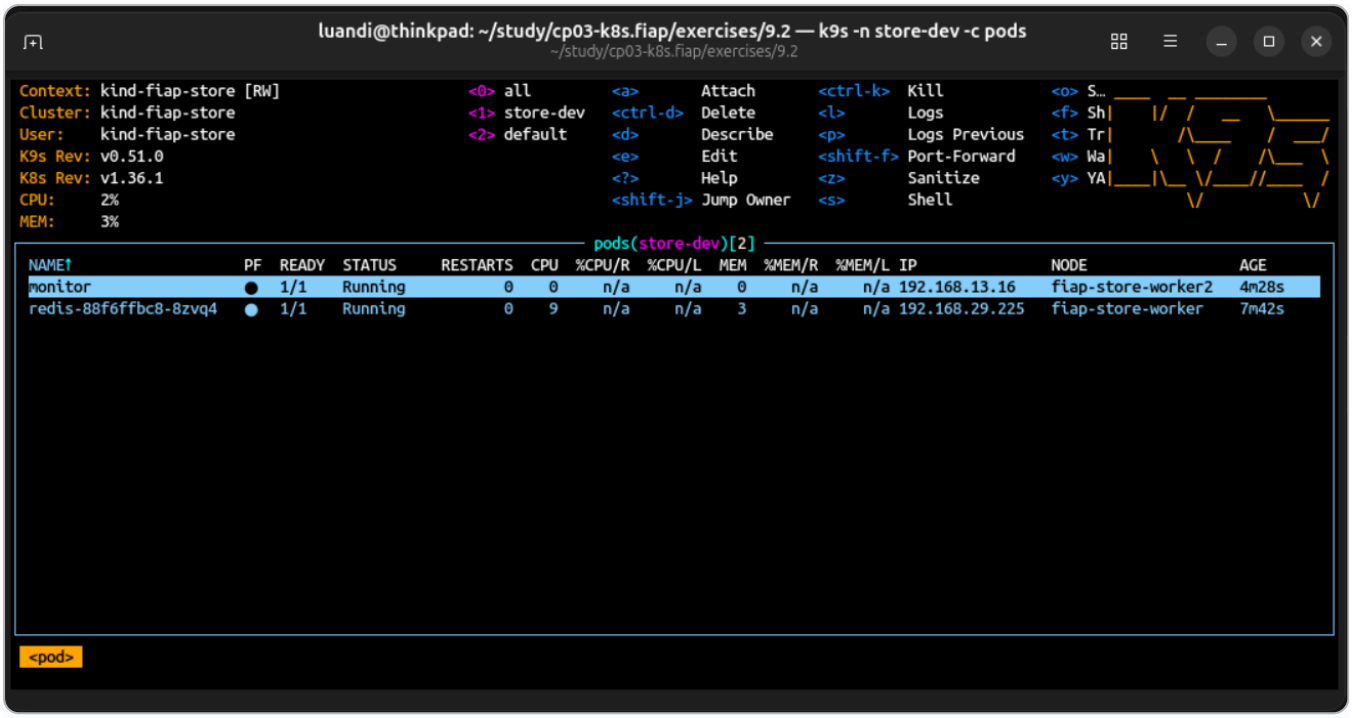
(c) Chamadas à API de dentro do Pod

```
$ kubectl -n store-dev exec monitor -- sh -c '
  SA=/var/run/secrets/kubernetes.io/serviceaccount
  curl -sS --cacert $SA/ca.crt -H "Authorization: Bearer $(cat $SA/token)" \
       https://kubernetes.default.svc/api/v1/namespaces/store-dev/pods'
HTTP 200
"kind": "PodList"
(2 Pods: monitor e redis-88f6ffbc8-8zvq4)

$ (mesmo comando em .../namespaces/store-dev/secrets)
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "secrets is forbidden: User \"system:serviceaccount:store-dev:monitor-sa\" cannot list
resource \"secrets\" in API group \"\" in the namespace \"store-dev\"",
  "reason": "Forbidden",
  "details": {
    "kind": "secrets"
  },
  "code": 403
}
HTTP 403
```

Foi usado `--cacert` com o `ca.crt` montado em vez do `-k` da dica. Assim o Pod verifica que está falando com o API server verdadeiro, sem desligar o TLS.

Observação sobre o token montado: o token que o kubelet monta no Pod não é o de 1 h do item (b). Decodificado, o `exp` dele é **24/10/2027**, um ano depois. O Pod declara `TokenExpirationSeconds: 3607`, mas o API server estende a validade para manter compatibilidade com clientes antigos que não releem o token (`--service-account-extend-token-expiration`, ligado por padrão). O kubelet continua renovando o arquivo antes da hora.



Critérios de aceite

Critério	Resultado
`can-i --list` mostra só os recursos concedidos	ok (5 recursos da Role mais os padrões de todo usuário autenticado)
`get secrets` e `create pods` retornam `no`	ok
`sub` do JWT igual a `system:serviceaccount:store-dev:monitor-sa`	ok
`curl` em `/secrets` com HTTP 403 e mensagem `forbidden`	ok